

Anomaly Detection in Large-Scale Cloud Systems: An Industry Case and Dataset

Mohammad Saiful Islam*, Mohamed Sami Rakha*, William Pourmajidi*, Janakan Sivaloganathan*,
John Steinbacher†, and Andriy Miranskyy*

*Dept. of Computer Science, Toronto Metropolitan University, Toronto, Canada,
Email: {mohammad.s.islam, rakha, william.pourmajidi, jsiva, avm}@torontomu.ca

†Cloud Platform, IBM Canada Lab, Toronto, Email: jstein@ca.ibm.com

Abstract—As Large-Scale Cloud Systems (LCS) become increasingly complex, effective anomaly detection is critical for ensuring system reliability and performance. However, there is a shortage of large-scale, real-world datasets available for benchmarking anomaly detection methods.

To address this gap, we introduce a new high-dimensional dataset from IBM Cloud, collected over 4.5 months from the IBM Cloud Console. This dataset comprises 39,365 rows and 117,448 columns of telemetry data. Additionally, we demonstrate the application of machine learning models for anomaly detection and discuss the key challenges faced in this process.

This study and the accompanying dataset provide a resource for researchers and practitioners in cloud system monitoring. It facilitates more efficient testing of anomaly detection methods in real-world data, helping to advance the development of robust solutions to maintain the health and performance of large-scale cloud infrastructures.

I. INTRODUCTION

In recent decades, the adoption of Cloud Computing across government and business sectors has grown exponentially [1], [2]. At the core of this growth is the ability of cloud computing to offer high-capacity data centers as a reliable backbone for services. Cloud providers operate expansive data centers that support global workloads, necessitating sophisticated techniques to monitor, diagnose, and respond to failures in real-time. As cloud infrastructure expands in both scale and complexity, maintaining its reliability has become a critical concern. Even brief outages or performance issues can lead to significant losses for users hosting applications in the cloud [3].

To prevent such issues, cloud system administrators must continuously monitor hardware and software services to ensure compliance with service-level agreements (SLAs) [4]. System anomalies, which translate into unexpected behavior, reduced efficiency, or even downtime, pose a significant risk. Early detection of these anomalies is vital for taking preemptive measures to safeguard users, improve the overall user experience, and ensure SLAs. Various studies have introduced anomaly detection methods based on statistical and machine learning techniques, spanning supervised [5], semi-supervised [6], [7], and unsupervised approaches [8]–[11]. These models have been tested using diverse datasets and systems of varying complexity and scale [12], [13].

One major challenge for anomaly detection methods is the high dimensionality of data generated in large-scale cloud

computing environments [14]. Many existing methods struggle to maintain accuracy in the presence of this “curse of dimensionality” [15], which hampers both performance and precision. High-dimensional data requires more input for generalization and results in data sparsity, where data points become scattered and isolated. The abundance of irrelevant features often obscures true anomalies, reducing the effectiveness of traditional methods such as distance or clustering-based techniques [16].

Additionally, much of the existing work in anomaly detection has been conducted on relatively small datasets [7]–[19], which may not fully capture the challenges posed by larger-scale cloud systems. To help advance this area of research, we aim to share a large-scale dataset from a real-world IBM Cloud System [9] with the broader community. This will enable more comprehensive testing and evaluation of anomaly detection methods on large, complex datasets. We address the following research questions (RQs):

RQ1: What are the key characteristics of telemetry datasets collected from Large-Scale Cloud Systems¹ (LCS)?

RQ2: What are the main challenges in predicting anomalies within such large datasets?

The main contributions of this paper are:

- Introducing a new large-scale dataset for testing anomaly detectors in cloud systems. The dataset is available on Zenodo [20].
- Demonstrating predictive models for anomaly detection in cloud environments. The reproducibility package is accessible on GitHub [21] and Zenodo [22].
- Discussing challenges related to handling high-dimensional telemetry data using domain knowledge and machine learning techniques.

The remainder of the paper is organized as follows. Section II reviews the related literature. Section III introduces the dataset. Section IV details the construction of anomaly detectors. Section V discusses the challenges faced. Finally, Section VII concludes the paper.

¹Comprising numerous hardware and software components, which are often distributed across multiple data centers.

II. RELATED WORK

A. Existing Datasets and Benchmarks

Listed below are popular datasets and benchmarks for detecting anomalies in Cloud systems.

The *NAB (Numenta Anomaly Benchmark) dataset* [17], [23] comprises 57 one-dimensional time series collected from diverse sources such as web traffic, power consumption, and sensor readings [24]. It includes both real-world and synthetic data intended for testing anomaly detection in real-time streaming applications. Each dataset contains an average of 6303 rows, ranging from 1127 to 22 695 rows. While NAB is recognized for its utility in real-time anomaly detection, it has technical weaknesses such as missing values and varying data distributions, limiting its practical utility for high-dimensional anomaly detection tasks [25].

The *Microsoft Cloud Monitoring dataset* consists of 67 one-dimensional time series from production telemetry signals [18]. Each dataset includes a timestamp, metric value, and anomaly label fields, capturing metrics like database query rates, service API latency, and application crash rates at per-minute or per-hour intervals. The datasets average 3757 rows, ranging from 176 to 20 160 rows. Although useful for evaluating anomaly detection algorithms, its small scale and low dimensionality limit its suitability for modeling complex relationships in high-dimensional cloud telemetry environments.

The *Exathlon dataset* is a high-dimensional dataset constructed from real data traces of repeated executions of large-scale stream processing jobs on an Apache Spark cluster over 2.5 months [26]. It includes 93 traces (after pruning) from 100 executions of 10 distributed streaming jobs, with six types of intentionally introduced anomalies like misbehaving inputs, resource contention, and process failures. Each trace consists of 2283 metrics recorded every second for about seven hours on average. While Exathlon serves as a benchmark for explainable anomaly detection in high-dimensional time series and is larger in scale and dimensionality than NAB, it focuses on a fixed set of repeated streaming tasks, which may limit its applicability for capturing the complexities and variability of dynamic cloud environments.

We will compare these datasets with ours in Section III-E.

B. Anomaly Detection Methods

We categorize the anomaly detection methods as follows.

a) Supervised Methods: Supervised approaches treat the anomaly detection problem as binary classification. With complete and accurate ground truth labels, supervised classifiers can detect known anomalies but may miss unknown ones. Typically, existing classifiers like Neural Networks [27] and Random Forest [28] are employed. A key challenge is that ground truth labels may not cover the full spectrum of anomaly types, limiting the ability of supervised methods to identify unfamiliar or unlabeled anomaly patterns [29].

b) Semi-Supervised Methods: Semi-supervised anomaly detection algorithms leverage partially available labels while retaining the capability to detect unseen anomalies. Recent studies use partially labeled data to enhance detection accuracy and exploit unlabeled data for representation learning. Some semi-supervised models are trained exclusively on normal samples, identifying anomalies that deviate from learned normal representations [29], [30].

c) Unsupervised Methods: Unsupervised anomaly detection methods operate under various assumptions about data distribution [31], such as anomalies residing in low-density regions. Their performance depends on how well the input data aligns with these assumptions. Numerous unsupervised methods have been proposed [32], broadly categorized into shallow and deep (neural network) methods. Shallow methods often offer greater interpretability, while deep learning (DL) methods excel with large, high-dimensional data. Unsupervised DL methods for anomaly detection in multivariate time series have gained significant attention due to the increasing complexity and dimensionality of software systems monitoring.

Unsupervised DL methods can be categorized along three key dimensions: (1) inter-variable correlation evaluation [33], (2) temporal context modeling [34], and (3) anomaly score criteria [35]. The first dimension involves methods for quantifying correlations among multiple variables, such as dimensional reduction, 2D matrices, or graphs [36], allowing high-dimensional monitoring data to be succinctly represented with reduced feature sets, mitigating dimensionality issues and computational resource requirements. The second dimension focuses on the temporal context within time series data, depending on the choice of neural network architectures like Recurrent Neural Networks (RNNs) [37], Long Short-Term Memory (LSTM) [38], Gated Recurrent Units (GRUs) [39], or Convolutional Neural Networks (CNNs) [40]. The third dimension revolves around determining anomaly scores that indicate levels of anomalousness; higher scores suggest a greater likelihood of abnormal behavior. Anomaly score calculations include methods like reconstruction error [41], prediction error [34], or dissimilarity metrics [42].

In our prior work [9], [10], we proposed an anomaly detection architecture based on a GRU-based autoencoder with a likelihood function for anomaly detection in multi-dimensional cloud telemetry. Our initial results showed that this model could detect anomalies up to 20 minutes earlier than previous monitoring systems and significantly reduced false alerts. The detector's performance was validated against both publicly available benchmark datasets and real-world data from the IBM Cloud Platform. That work focused on an earlier version of the software under study, which has significantly evolved over the past few years, becoming more sophisticated yet increasingly complex (in line with the Laws of Software Evolution [43]). Since 2021, the IBM Cloud Platform has undergone significant changes, including migration to containerized environments and the implementation of new technologies, leading to data with new characteristics that require tailored anomaly detection methods to address

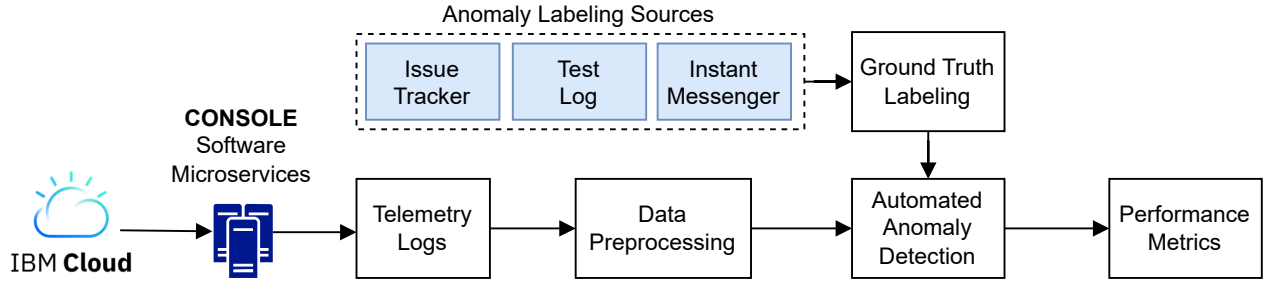


Fig. 1. Overview of the data pipeline.

evolving challenges effectively.

In this paper, we aim to highlight the challenges raised by the IBM Cloud Platform’s evolved system, particularly in working with the new data it generates. Unlike the previous studies [9], [10], which did not share the data or models, this work provides both, enabling the community and industry to adopt the dataset as a benchmark for detection of anomalies in operation of large-scale anomaly cloud software. Additionally, we examine the difficulties in identifying ground truth labels and constructing automatic anomaly detection techniques on these complex datasets.

III. DATASET CREATION AND DESCRIPTION

A. Software System Under Study

In this study, we collected data from the IBM Cloud Console (hereon referred to as the CONSOLE), the primary web interface and orchestrator² for IBM Cloud [10]. IBM Cloud is a public cloud infrastructure with a global network of over 60 data centers [44]. Figure 1 presents an overview of the data pipeline in this study, illustrating the process from data extraction, through model training, to performance evaluation.

The CONSOLE production software system is deployed across seven data centers worldwide, though not all instances are active at the same time; they are rotated based on operational requirements. Additional CONSOLE deployments are spread globally for testing and staging environments. The CONSOLE uses a microservices architecture, with each microservice generating millions of daily logs and telemetry records (a common challenge with large scale Cloud systems [45]–[48]). These records offer valuable insights into the health and performance of the IBM Cloud, providing critical data for monitoring and optimization. The sheer volume and variability of logs and telemetry, combined with the dynamic nature of containerized environments — characterized by transient workloads, volatile resource usage, and continuous scaling — necessitate advanced, context-aware anomaly detection techniques. Traditional static or threshold-based methods fall short in addressing the complexity and scale of these systems. Our anomaly detection approach aims to address these challenges by leveraging innovative techniques designed

for dynamic environments, contributing to more effective monitoring and actionable insights.

B. Data Collection Methodology

In this paper, we collected telemetry data from the IBM Cloud CONSOLE, including logs and metrics generated by its microservices. Due to the large volume of logs and telemetry emitted by the microservices, we used a Publish/Subscribe (Pub/Sub) mechanism [49] to efficiently manage the data collection. The microservices publish their logs through a Redis Pub/Sub system [50] as part of this process.

We developed a resilient pipeline for the collection and analysis of real-time logs from Redis Pub/Sub, using our previous research [10], [51], [52]. This pipeline is data-agnostic, capable of receiving, processing, and storing data in the IBM Cloud Object Store (COS) [53] for near-real-time analysis.

A sub-pipeline called “Firehose” subscribes to Redis Pub/Sub, continuously receiving log data in Zipkin format³ [55]. Our containerized microservices, managed by Kubernetes, connect to Firehose to perform Extract, Transform, and Load operations, after which the processed data are stored in IBM COS. These microservices are managed by IBM DevOps tools [56] and toolchains [57] within a Continuous Integration/Continuous Delivery framework.

In this study, we collected a large dataset over ≈ 4.5 months — from January 22, 2024, to June 7, 2024. The data comes from seven production data centers during this period, providing a comprehensive view of the CONSOLE system’s performance.

C. Dataset Description

The collected CONSOLE dataset provides response time information for individual requests processed by software microservices, aggregated over 5-minute intervals. The choice of a 5-minute interval was based on feedback from the IBM Operations (Ops) team to reduce noise and improve data usability. The response time aggregation was performed using eight statistical functions: minimum, maximum, median, average, count, standard deviation, skewness, and kurtosis.

The final tabular dataset contains a total of 39 365 rows, each row representing a 5-minute interval. The dataset includes

²The CONSOLE supports key functions like identity management, billing, search, tagging, and access to the product catalog.

³At the time of writing, this sub-pipeline was upgraded to receive data in Open Telemetry format [54].

one column for the start time of each interval and 117448 columns for the aggregated statistics. The column names specify details such as the datacenter, host microservice endpoint, request type, and response code. Details on data preprocessing are provided in Appendix A. The characteristics of the dataset are provided in Appendix B.

D. Annotation Process for Anomalies

1) *Anomaly Labelling Sources*: We created ground-truth labels for anomalies using data from three sources, including an issue-tracking system, test log monitoring, and an internal instant messaging platform [58] (see Figure 1). Each source offered a distinct perspective on system anomalies, enabling us to construct a comprehensive and accurate representation of the software system issues.

a) *Issue Tracker*: This tracker focuses on customer-impacting events (such as disruptions affecting customer experience, service access, or quality). Typically, these issues were followed by root cause analysis, which provide additional insights into their origins.

b) *Test Log*: This tracker recorded alerts triggered by failures in synthetic UI test cases or heartbeat signal disruptions. Incidents were logged when failure thresholds were exceeded to minimize false alarms, signaling potential system issues.

c) *Instant Messenger*: We also monitored IBM Ops instant messaging communications, where potential anomalies were informally discussed. Only incidents confirmed by the IBM Ops team were included in the ground truth labels.

By integrating data from these three sources, we ensured that the ground-truth labels represented a comprehensive view of system anomalies. *In total, we identified 3 anomalies from the Issue Tracker, 11 anomalies from the Test Log, and 11 anomalies from the Instant Messenger.*

2) *Anomaly Timing*: The start and end times of anomalies were estimated based on expert guidance from the IBM Ops team. The start time was set 20 minutes before the issue was reported, reflecting the typical delay between detection, initial triage, and the creation of a ticket or instant message. End times were derived from duration data recorded in the Issue Tracker, Testing Log, or Instant Messenger based on human confirmation of issue resolution.

From our discussion with the IBM Ops team, we learned that some anomalies were mitigated by measures such as redirecting traffic to healthy instances as a high availability strategy. This approach often allowed anomalies to persist on certain DC instances for extended periods (e.g., hours) without noticeably affecting the user experience. However, the IBM Ops team was still responsible for identifying and resolving the underlying root causes of these anomalies to ensure long-term system stability.

E. Comparison with Existing Datasets

The existing benchmark datasets (as discussed in Section II-A) are widely used for comparing anomaly detection models in cloud environments. However, they often have

TABLE I
COMPARISON OF DATASETS FOR ANOMALY DETECTION IN CLOUD SYSTEMS.

Dataset Name	File Count	Column Count	Avg. Row Count	Row Range
NAB	57	1	6303	1127–22 695
MS Cloud Monitoring	67	1	3757	176–20 160
Exathlon	93	2283	25 200	25 200
Ours	1	117 449	39 365	39 365

limitations such as low dimensionality, with feature counts ranging from 1 to 2283, reliance on synthetic data, and a focus on static tasks (for Exathlon). These constraints reduce their relevance for the dynamic and large-scale nature of real-world cloud systems.

In contrast, our dataset is distinguished by its use of live data⁴, extended monitoring period (≈ 4.5 months), high dimensionality ($\approx 1.1 \times 10^5$), and focus on a real cloud environment (from IBM Cloud). These factors make it more suitable for analyzing and predicting anomalies in large-scale cloud-based systems, where capturing interactions between various components over time is crucial in large systems. The key statistics of the datasets are summarized in Table I.

IV. AN EXAMPLE OF DETECTING ANOMALIES

A. Predictive Models

To demonstrate how to use the collected dataset for anomaly prediction and model building, we present two simple autoencoders: one based on an artificial neural network (ANN) using Multi-Layer Perceptrons [59], and another based on the GRU [39] architecture. The former is simpler, while the latter is more sophisticated. Both models are implemented using the TensorFlow package v.2.15.0 [60]. The architectures of the autoencoders are as follows.

1) *ANN Autoencoder*: The ANN Autoencoder is a fully connected model well suited for feature-based anomaly detection. It starts with an input layer of 2410 dimensions, followed by a series of dense layers that gradually compress the data down to a latent space of 14 neurons. The encoding path reduces the dimensionality with layers of 128 and 64 neurons, each followed by Leaky ReLU activations [61], batch normalization, and dropout for regularization. The decoder then mirrors the encoding structure, expanding back to 2410 dimensions. The model contains $\approx 6.4 \times 10^5$ trainable parameters.

2) *GRU Autoencoder*: The GRU Autoencoder is generally designed for sequential data. It processes our 2410 featured sequential data, with seven stacked GRU layers compressing the input into a latent space of 14 features. Each GRU layer has 16 units with ReLU [62] as an activation function, which captures temporal dependencies in the data. After encoding, the latent features are expanded through a repeat vector layer,

⁴The CONSOLE microservices handled $\approx 3.2 \times 10^9$ requests during the 4.5 month period (based on the sum of all aggregated_stats_value fields where aggregated_stats_name is equal to count).

allowing the decoder to reconstruct the original sequence. The decoder uses seven GRU layers to gradually reconstruct the sequence back to its original length. The model contains $\approx 1.8 \times 10^7$ trainable parameters.

3) *Anomaly Likelihood Function*: For both models, the reconstruction error is passed to the anomaly likelihood function introduced in [17], [23]. The likelihood function is constructed as follows. It maintains a window of the last W error values and processes raw errors incrementally. Historical errors are modeled as a rolling normal distribution of a window of the last W points at each step t . The empirical mean μ_t and standard deviation σ_t at time t are computed as follows:

$$\mu_t = \frac{\sum_{i=0}^{W-1} s_{t-i}}{W}, \quad (1)$$

where $s_{(\cdot)}$ is the prediction error computed by the model, and

$$\sigma_t = \sqrt{\frac{\sum_{i=0}^{W-1} (s_{t-i} - \mu_t)^2}{W - 1}}. \quad (2)$$

Similarly to Eq. 1, we compute the empirical mean for a moving window W' , deemed $\tilde{\mu}_t$. By design $W' \ll W$; i.e., W and W' are long- and short-term intervals, respectively.

The likelihood of anomaly at time t , deemed L_t , is

$$L_t = 1 - Q\left(\frac{\tilde{\mu}_t - \mu_t}{\sigma_t}\right), L_t \in (0, 1), \quad (3)$$

where Q is a Gaussian tail probability [63]. For a user-defined threshold ϵ , if $L_t \geq 1 - \epsilon$, an observation at time t is classified as anomalous.

B. Experimental Setup

1) *Feature Preparation and Dataset Split*: In the dataset features preparation, we kept only the subset of features corresponding to HTTP return codes in the 5XX range (server errors) and the “count” aggregate statistical function, resulting in a total of 2406 features. This step was done to reduce the dataset size based on the assumption that anomalous behavior is often reflected in changes in the frequency of server errors. To validate this assumption, we plot the distribution of the count of requests associated with 5XX codes in Figure 2. This distribution was obtained by summing all relevant features for each row, providing a cumulative error count for each timestamp. The right-skewed distribution, with a few timestamps exhibiting very high counts, suggests that anomalies are present in the data, making them a point of interest for anomaly detection purposes.

To capture weekly and daily periodicity, we added seasonality features using sine and cosine trigonometric functions [64]. This resulted in 2410 features in our input. The training data were scaled using min-max normalization, and the same normalization parameters were applied to the test data. Null values were replaced by zeros. The dataset was divided into five weeks of training data (from 2024-01-26 to 2024-02-29) and three months of test data (from 2024-03-01 to 2024-05-31). The test data contain 19 anomalies.

Each observation in the model represents data collected over an individual 5-minute interval.

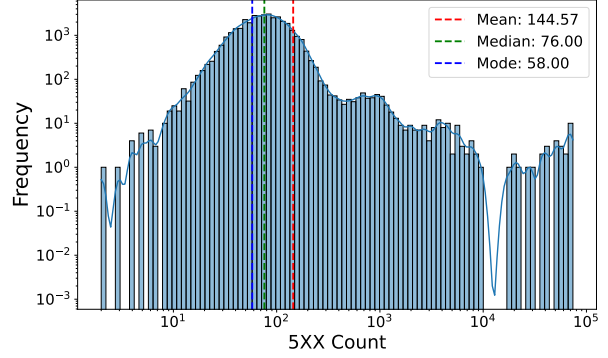


Fig. 2. The distribution of request counts associated with 5XX errors. Both the x -axis and y -axis are logarithmically scaled, with the x -axis representing the number of 5XX errors and the y -axis showing the frequency of each corresponding error count. The solid blue line illustrates the overall distribution, highlighting the concentration and variation across different 5XX error values.

2) *Evaluation Metrics*: We evaluated our models using the standard confusion matrix metrics: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) [65], [66]. In this context, the positive label represents an anomaly, and the negative label denotes a non-anomaly.

However, in anomaly detection, relying solely on these point-based metrics is not ideal because detecting just one instance within an anomaly window is often sufficient in practice [23]. To address this, in addition to the conventional metrics, we also employed a window-based metric from the Numenta Anomaly Benchmark (NAB) to provide a more realistic evaluation of the models’ performance. This metric is known as the *NAB score*, ranging from 0 to 100, with higher numbers indicating better performance. Although not perfect [25], the NAB score is more practical for this task than metrics based on confusion matrices, such as accuracy or F1-score [65].

For a detailed explanation of the NAB score, refer to [17], [23]. In brief, this metric rewards early detection (TP) within an anomaly window and penalizes false positives and false negatives. Any detection within an anomaly window is considered a TP, with its score determined by a scaled sigmoid function. A detection at the start of the anomaly window is assigned a score of 1, while detections near the end receive lower scores, closer to 0. Any subsequent detections within the same window are ignored for scoring purposes, meaning that all detections within one window count as a single TP. If the model misses an entire anomaly window, it results in one FN. FP and TN, on the other hand, are evaluated point-by-point. TNs do not affect the NAB score, but each FP lowers the score.

The NAB score can be computed using different cost profiles; we use two shown in Table II: “Standard” and “Reward Low FN”. The “Standard” profile strikes a balance between TP, FN, and FP, applying a modest penalty on FP to prevent the model from favoring either precision or recall too

TABLE II
NAB SCORE COST PROFILES AS PER [17].

Profile	TP Weight	FN Weight	FP Weight	TN Weight
Standard	1.00	1.00	0.11	1.00
Reward Low FN	1.00	2.00	0.11	1.00

heavily. In contrast, the “Reward Low FN” profile applies a harsher penalty on FN, which is useful in scenarios like ours where detecting each anomaly is crucial due to the importance of anomalies present in the dataset. This higher penalty on false negatives encourages the model to prioritize finding the anomalies, even at the risk of increasing FP count.

3) *Training and Testing Process*: We trained the autoencoder on the training data. For the anomaly likelihood function, we set $L_t = 0.9996$, $W = 30$, and $W' = 2$; these values were chosen based on our prior experience with this type of task⁵. The trained model was then tested on the test dataset.

C. Results

1) *Quantitative Results*: Table III presents the performance of the models in detecting anomalies.

The GRU model showed a slight improvement over the ANN model. The NAB score increased from 4.61 to 6.06 for the “Standard Profile” and from 4.61 to 13.60 for the “Low FN Profile.” However, the marginal improvements observed with the GRU model were somewhat unexpected, suggesting potential for further enhancement through hyperparameter tuning. Techniques such as advanced optimization strategies or refinements to the model architecture could better harness the GRU’s capabilities and address current limitations in the experimental setup.

Both models detected 1 out of 3 customer-impacting anomalies from the Issue Tracker, 5 out of 9 anomalies from the Instant Messenger threads, and none of the 7 anomalies from the Test Log.

Figure 3 illustrates an example of anomalies captured by the GRU autoencoder. The figure highlights both the anomalies the model successfully detects and those it misses. It also points out certain spikes in the number of 5XX errors that, despite being obviously abnormal, are not flagged as anomalies. There are several reasons for this. For example, some anomalies may not directly affect the CONSOLE, while others are mitigated automatically through techniques such as high availability or caching.

2) *Qualitative Analysis*: We will highlight two example cases observed in the test data. The first case involves a true operational issue successfully detected by the model. The second case, although flagged as anomalous by the model,

⁵Specifically, we conducted a small grid search using the following hyperparameter values: $L_t \in \{0.9990, 0.9995, 0.9996, 0.9997, 0.9998\}$, $W \in \{20, 25, 30, 35, 40, 50\}$, and $W' \in \{1, 2, 3, 4, 5\}$. While these represent a sample of potential model configurations, further performance improvements could be achieved with more extensive hyperparameter tuning. The chosen models and parameters are intended as illustrative examples, rather than as optimal configurations.

shows unusual system behavior that does not correspond to any known issues in the ground truth data.

a) *Case 1 (True Positives)*: During the period of March 12-13, 2024 (Figure 4), the GRU-based anomaly detection model successfully flagged two anomaly windows. Both required taking specific instances of CONSOLE offline and redirecting traffic to healthy instances. The model not only detected these extended anomaly windows but also identified the second anomaly early, demonstrating the model’s ability to proactively catch issues before they escalate.

b) *Case 2 (False Positives)*: Figure 5 illustrates abnormal behavior detected on April 15-18, 2024, where the 5XX error count spiked to the highest level observed during the study. The spike lasted for nearly two hours and was accompanied by a sharp rise in reconstruction error, suggesting a significant deviation from normal patterns. The anomaly likelihood remained elevated throughout the event, which led the GRU model to flag it as anomalous.

While both cases present clear signs of abnormality (high error spikes and extended durations), only the first case corresponds to an actual operational issue.

Overall, these case examples highlight the challenges of modeling LCS behavior. Although the GRU model successfully identified unusual behavior, it underscores the need for further refinement to distinguish between true operational issues and abnormal patterns that do not require human intervention. Incorporating human feedback, for example, through a human-in-the-loop training model [10], [67]–[69], could be essential in reducing false positives and improving the model’s reliability in real-world applications.

D. Interpretation of Results

A test of commonly used anomaly detection models shows low performance, with only 6 anomalies detected out of 19. Additionally, around 70 abnormalities were identified in the system (similar to those described in Section IV-C2b), but these may not be useful to the IBM Ops team, as they could be resolved by various automatic mechanisms. *This highlights the challenge of accurately detecting anomalies in the LCS and calls on the community to develop more sophisticated models to address this issue.*

V. INSIGHTS AND CHALLENGES

Our **RQ1** was: “What are the key characteristics of telemetry datasets collected from LCS?” As discussed earlier, our dataset exhibits high dimensionality and volatility, reflecting the dynamic nature of LCS — complex, evolving systems with fluctuating workloads. These characteristics present significant challenges for anomaly prediction, leading us to **RQ2**: “What are the main challenges in predicting anomalies within such large datasets?” Below, we explore some of these challenges and the corresponding insights.

The size of the system and the corresponding large feature set (on the order of 10^5) make model training computationally expensive. This, in turn, complicates model hyperparameter tuning due to the high resource demands. For example, during

TABLE III
PERFORMANCE OF THE ANOMALY DETECTORS. AE DENOTES AUTOENCODER.

Model	Confusion Matrix				NAB Score		Count of Detected Anomalies		
	TP	TN	FP	FN	Standard Profile	Low FN Profile	Issue Tracker	Instant Messenger	Test Log
GRU AE	8	25 450	71	959	6.06	14.57	1 out of 3	5 out of 9	0 out of 7
ANN AE	8	25 445	76	959	4.61	13.60	1 out of 3	5 out of 9	0 out of 7

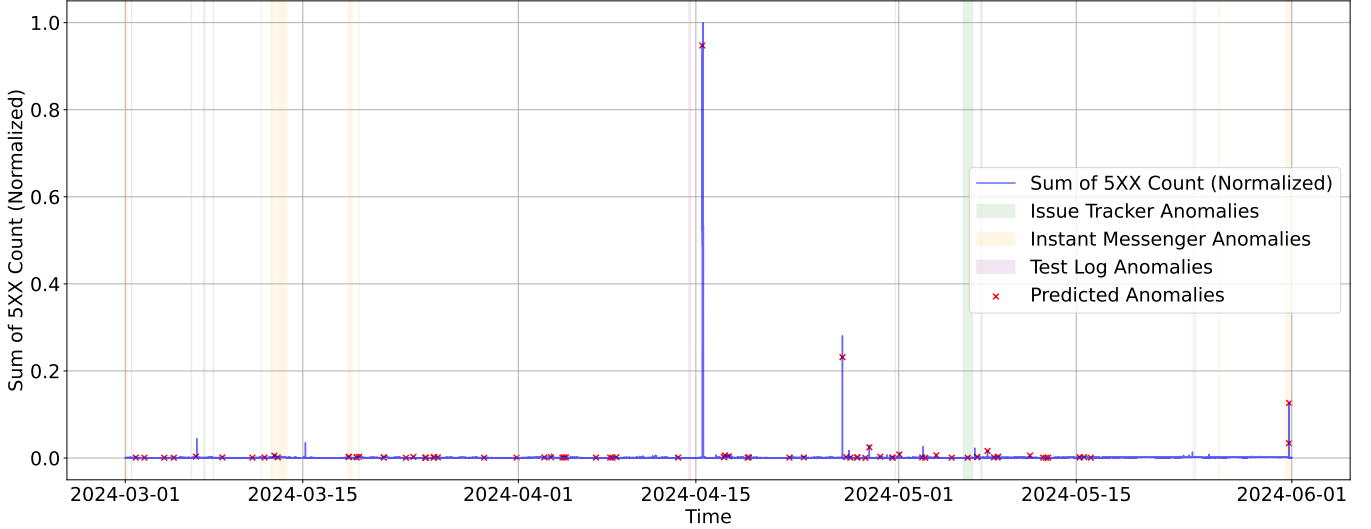


Fig. 3. An example of anomaly detection using the GRU autoencoder. The x -axis represents the observation time. The y -axis shows (for simplicity of comprehension) the normalized sum of 5XX return codes, depicted by the blue line. Detected anomalies are marked with red crosses, while true anomaly windows are indicated by vertical bars in different colors, corresponding to the different sources of the anomaly reports.

our experiments on the complete dataset, hyperparameter tuning for the trained models demanded significant computational resources, often resulting in prolonged training times and system bottlenecks, such as memory exhaustion or CPU overload during the deployment of anomaly detection systems. One possible solution is to reduce the dimensionality, either by selecting salient features based on expert knowledge or by employing automatic dimensionality reduction techniques. Developing scalable algorithms that balance computational efficiency and model performance remains a critical area for future work.

The IBM Cloud system we study is inherently non-stationary: the hardware and software stacks are constantly evolving, new features are regularly introduced, and the number of active users and their workloads fluctuate over time. Given the complexity of this system, unusual activities and failures often occur. However, the ground-truth data may not always reflect these activities and failures (see Section IV-C2b).

As a result, anomaly detectors may flag issues that appear to be false positives when compared to ground-truth but are, in fact, genuine deviations from normal behavior. An open challenge is determining how to automatically distinguish between two types of anomalies: those that impact customers

versus those that are handled and resolved automatically⁶. Being able to make this distinction would reduce the burden on operations teams, particularly when triaging alerts during off-hours, which can be exhausting.

An alternative potential solution to address non-stationary is to incorporate data from multiple 5-minute intervals into each observation to capture more complex patterns and regularly retrain the models to address non-stationarity (similar to the approach in [10]). However, this poses a challenge due to the high computational costs involved.

Another challenge is accurately identifying the start and end times of anomalies in such a complex system (as discussed in Section III-D). This is a known issue in real-world datasets (especially those having a high number of features) [17]. Although we attempt to identify the time windows of the anomalies, they are not always perfectly precise. This imprecision makes it harder for models to distinguish between normal and abnormal behavior. One approach is to introduce a “time buffer” around anomalous windows to prevent abnormal be-

⁶The latter type of anomalies — those that are automatically mitigated — are typically lower priority. However, they may still signal areas for potential system refactoring to improve robustness. This presents a common requirements prioritization dilemma: Should resources be allocated toward the development of new features or to reducing the number of failover events that are automatically mitigated and do not impact customers?

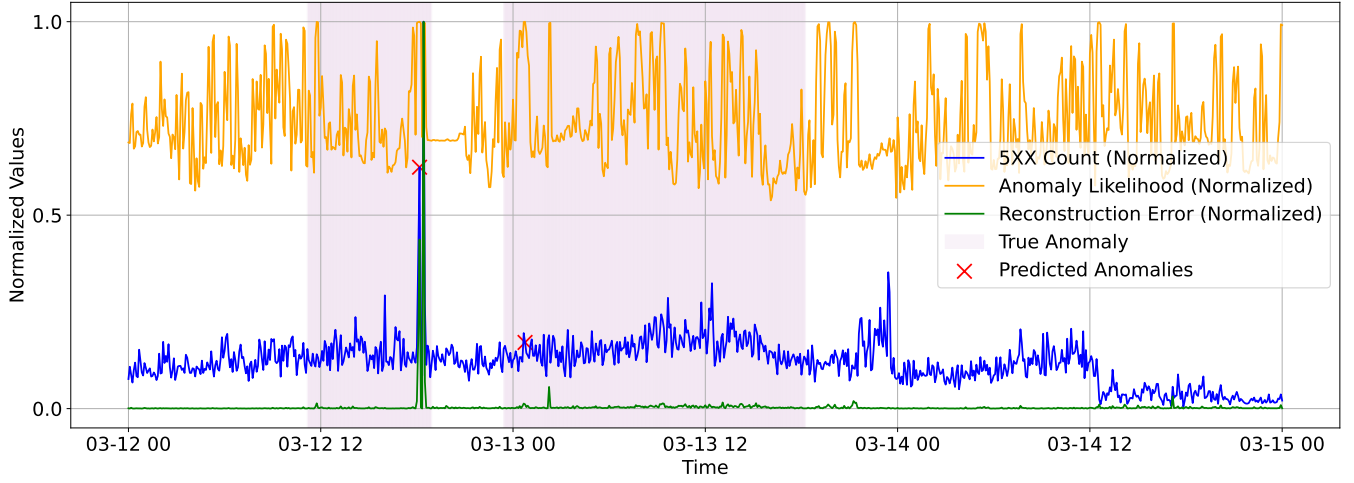


Fig. 4. Example of two subsequent anomaly detections (present in the ground truth file) on March 12-13, 2024, using the GRU autoencoder model. The detected anomalies are highlighted with red crosses, while true anomaly windows are marked with purple bars. The blue line represents the normalized sum of the 5XX return codes count, the green line shows the reconstruction error, and the orange line indicates anomaly likelihood.

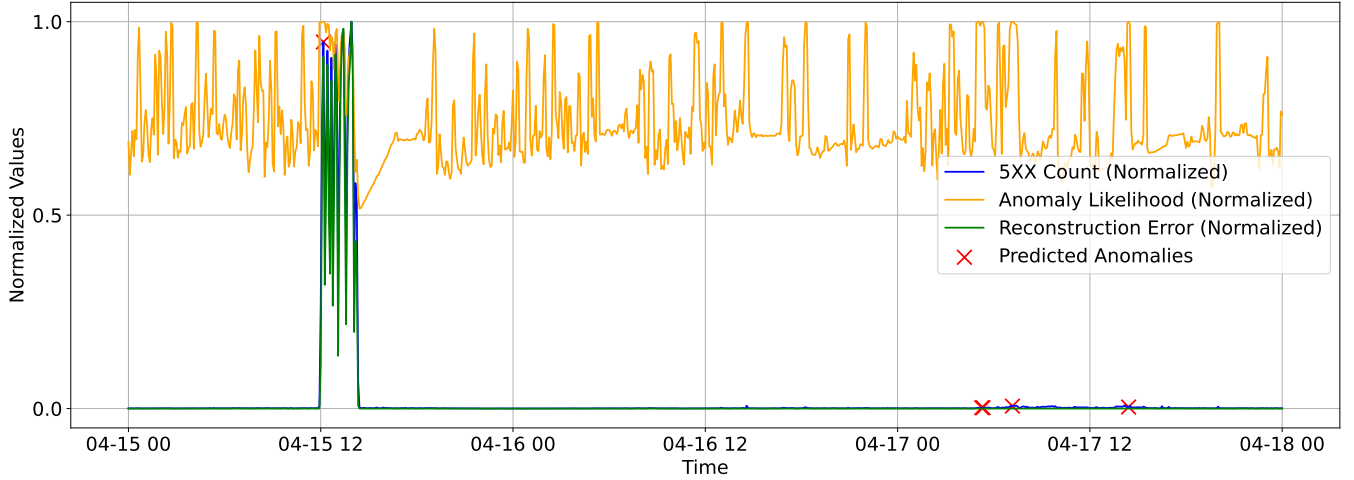


Fig. 5. Example of detected abnormal behavior (absent from the ground truth file) from April 15-18, 2024, using the GRU autoencoder model. The blue line represents the normalized sum of the 5XX return codes count, the green line shows the reconstruction error, and the orange line indicates anomaly likelihood.

havior from slipping into the training dataset of autoencoders. Metrics such as the NAB score help mitigate this to some extent, as reporting false positives near the end of a previous anomaly window results in a lower penalty.

VI. THREATS TO VALIDITY

This study, while providing valuable insights into anomaly detection within large-scale cloud systems, faces several potential threats to validity that should be acknowledged. We classify validity threats according to [70], [71].

A. External Validity

Software engineering studies are often challenged by the variability of real-world environments, and the problem of *generalization* cannot be fully resolved [72]. In this work, we

consider our software and its associated telemetry dataset as a critical case⁷, which could assist the community in building anomaly detection models and testing them on the data from a real-world LCS. The same empirical evaluation and analysis methods can be applied to other software products, provided that well-designed and controlled experiments are conducted.

Beyond addressing the generalization problem, this dataset holds practical value for various stakeholders in cloud environments. For instance, if a client operates a complex cloud application (e.g., distributed and comprising multiple components), this dataset would remain relevant and of interest. Models trained at the provider level can help customers by identifying

⁷In case study research, a critical case refers to a particularly significant or decisive instance chosen for its potential to offer deep insights into a specific phenomenon [71].

patterns or anomalies specific to distributed workloads, thereby improving the robustness and reliability of their operations. This highlights the practical value of the dataset beyond cloud providers, extending its applicability to end-users managing sophisticated cloud-based applications.

B. Internal Validity

Our dataset, while extensive, covers a limited time span of 4.5 months and represents telemetry data from a specific subset of IBM Cloud services. As such, it may not capture longer-term trends, rare failure modes, or seasonal variations that could influence the behavior of large-scale cloud systems.

The trace data was system-generated, incorporating traces from both real users and synthetic test runs. However, the synthetic test cases used for generating telemetry data may not fully represent the complexity of real-world user interactions and may overlook some issues. Thus, the synthetic test runs (which resulted in 11 “Test Log” anomalies) could bias our anomaly detection results, potentially leading to an over-representation or under-representation of certain types of incidents that typically occur in live environments.

Despite these limitations, the 4.5-month time span remains valuable for identifying short-term patterns and providing insights into common issues.

C. Construct Validity

A challenge in this study is the difficulty of accurately defining ground truth for anomaly detection in LCS. For example, the actual start of an issue might be imprecise due to the reliance on thresholds or operational heuristics, and the termination of incidents might overshoot the real closing time. Additionally, some abnormal behavior may not affect end-users (e.g., due to high-availability mechanisms), further complicating the identification of true incidents. Despite these complexities, they reflect the realities of working with real-world datasets collected from complex systems.

VII. CONCLUSION

In this paper, we introduce a novel, large-scale dataset of real-world telemetry from IBM Cloud’s Console software, a continuously evolving LCS. Collected over 4.5 months, the dataset captures aggregated response-time telemetry from microservices across multiple data centers. We intend for this dataset to serve as a new benchmark challenge for researchers and practitioners developing anomaly detection methods.

Our experiments utilized two predictive models for anomaly detection — ANN-based and GRU-based autoencoders. While both showed potential, they also highlighted key challenges, including high data dimensionality, non-stationary behavior, and difficulty in distinguishing between significant and insignificant anomalies in cloud systems.

More advanced techniques are required to effectively detect and predict anomalies that affect customer experience and system stability. Future research could investigate more sophisticated machine learning models, dimensionality reduction strategies, and active learning approaches to better manage

the dynamic nature of cloud environments. Building a reliable anomaly detector that accurately predicts impactful anomalies remains an open research question, and we encourage the research community to contribute to this effort.

This study adds to the growing research on cloud anomaly detection by providing a benchmark dataset and addressing the practical difficulties of detecting anomalies in real-world cloud systems. We hope our dataset and findings will inspire the creation of more robust anomaly detection solutions, ultimately improving the reliability and performance of cloud services.

REFERENCES

- [1] R. Buyya, C. Vecchiola, and S. T. Selvi, *Mastering Cloud Computing: Foundations and Applications Programming*, ser. ITPro Collection. Morgan Kaufmann, 2013. [Online]. Available: <https://books.google.ca/books?id=wqKqHJhPJQC>
- [2] I. Gartner, “Gartner forecasts worldwide public cloud end-user spending to surpass \$675 billion in 2024,” Gartner Newsroom, 2024. [Online]. Available: <https://www.gartner.com/en/newsroom/press-releases/2024-05-20-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-surpass-675-billion-in-2024>
- [3] Z. Chkibene, A. Erbad *et al.*, “Machine learning based cloud computing anomalies detection,” *IEEE Network*, vol. 34, no. 6, pp. 178–183, 2020.
- [4] L. Wu, S. K. Garg, and R. Buyya, “Service level agreement (sla) based saas cloud management system,” in *2015 IEEE 21st International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2015, pp. 440–447.
- [5] M. Farshchi, J.-G. Schneider *et al.*, “Experience report: Anomaly detection of cloud application operations using log and cloud metric correlation analysis,” in *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*, 2015, pp. 24–34.
- [6] P. K. Deka, Y. Verma *et al.*, “Semi-supervised range-based anomaly detection for cloud systems,” *IEEE Transactions on Network and Service Management*, 2022.
- [7] F. Gao, J. Li *et al.*, “Connet: Deep semi-supervised anomaly detection based on sparse positive samples,” *IEEE Access*, vol. 9, pp. 67 249–67 258, 2021.
- [8] S. Baek, D. Kwon *et al.*, “Unsupervised labeling for supervised anomaly detection in enterprise and cloud networks,” in *2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE, 2017, pp. 205–210.
- [9] M. S. Islam and A. Miranskyy, “Anomaly detection in cloud components,” in *IEEE 13th International Conference on Cloud Computing (CLOUD)*, 2020, pp. 1–3.
- [10] M. S. Islam, W. Pourmajidi *et al.*, “Anomaly detection in a large-scale cloud platform,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2021, pp. 150–159.
- [11] G. Zhong, F. Liu *et al.*, “Detecting cloud anomaly via broad network-based contrastive autoencoder,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 3249–3263, 2024.
- [12] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey,” *ACM Computing Surveys (CSUR)*, vol. 55, no. 3, pp. 1–39, 2022.
- [13] T. Hagemann and K. Katsarou, “A systematic review on anomaly detection for cloud computing environments,” in *Proceedings of the 2020 3rd Artificial Intelligence and Cloud Computing Conference*, 2020, pp. 83–96.
- [14] S. Thudumu, P. Branch *et al.*, “A comprehensive survey of anomaly detection techniques for high dimensional big data,” *Journal of Big Data*, vol. 7, pp. 1–30, 2020.
- [15] R. Bellman, *Adaptive control processes: a guided tour*. Princeton University Press, 1961.
- [16] C. C. Aggarwal and P. S. Yu, “Outlier detection for high dimensional data,” in *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’01. New York, NY, USA: Association for Computing Machinery, 2001, p. 37–46. [Online]. Available: <https://doi.org/10.1145/375663.375668>

- [17] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark," in *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.
- [18] "Microsoft cloud monitoring dataset," 2023. [Online]. Available: <https://github.com/microsoft/cloud-monitoring-dataset>
- [19] M. Panahandeh, A. Hamou-Lhadj *et al.*, "Serviceanomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *Journal of Systems and Software*, vol. 209, p. 111917, 2024.
- [20] M. S. Islam, M. S. Rakha *et al.*, "Dataset for the paper "anomaly detection in large-scale cloud systems: An industry case and dataset"," 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.14062900>
- [21] —, "Icse seip 2025 artifact repository," Dec. 2024, accessed: 2025-01-04. [Online]. Available: <https://github.com/msi-ru-cs/icse-seip2025-anomaly-detector-public>
- [22] —, "Reproducibility package for "anomaly detection in large-scale cloud systems: An industry case and dataset"," Jan. 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.14598119>
- [23] S. Ahmad, A. Lavin *et al.*, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [24] "Numenta anomaly benchmark (nab)," 2024. [Online]. Available: <https://github.com/numenta/NAB>
- [25] N. Singh and C. Olinsky, "Demystifying numenta anomaly benchmark," in *2017 International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 1570–1577.
- [26] V. Jacob, F. Song *et al.*, "Exathlon: a benchmark for explainable anomaly detection over time series," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 2613–2626, Jul. 2021.
- [27] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [28] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [29] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," *arXiv preprint arXiv:1901.03407*, 2019.
- [30] L. Ruff, R. A. Vandermeulen *et al.*, "Deep semi-supervised anomaly detection," *arXiv preprint arXiv:1906.02694*, 2019.
- [31] V. J. Hodge and J. Austin, "A survey of outlier detection methodologies," *Artificial Intelligence Review*, vol. 22, no. 2, pp. 85–126, 2004.
- [32] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [33] L. Zhang, C. Wang, and G. Cottrell, "Deep metric learning for multivariate time series," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 5981–5990.
- [34] K. Hundman, V. Constantinou *et al.*, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 387–395.
- [35] B. Zong, Q. Song *et al.*, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [36] J. Lee and M. Verleysen, "Nonlinear dimensionality reduction," *Springer*, 2007.
- [37] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [38] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [39] K. Cho, B. van Merriënboer *et al.*, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.
- [40] Y. LeCun, L. Bottou *et al.*, "Gradient-based learning applied to document recognition," in *Proceedings of the IEEE*, vol. 86, no. 11, 1998, pp. 2278–2324.
- [41] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, pp. 4–11, 2014.
- [42] L. Ruff, J. Kauffmann *et al.*, "A unifying review of deep and shallow anomaly detection," *arXiv preprint arXiv:2009.11732*, 2020.
- [43] M. M. Lehman, "Laws of software evolution revisited," in *European workshop on software process technology*. Springer, 1996, pp. 108–124.
- [44] "Ibm cloud: Company profile, data center locations." [Online]. Available: <https://www.datacenters.com/providers/ibm-cloud>
- [45] A. Miranskyy, A. Hamou-Lhadj *et al.*, "Operational-log analysis for big data systems: Challenges and solutions," *IEEE Software*, vol. 33, no. 2, pp. 52–59, 2016.
- [46] W. Pourmajidi, J. Steinbacher *et al.*, "On challenges of cloud monitoring," in *Proceedings of the 27th Annual International Conference on Computer Science and Software Engineering*, 2017, pp. 259–265.
- [47] W. Pourmajidi, A. Miranskyy *et al.*, "Dogfooding: Using ibm cloud services to monitor ibm cloud infrastructure," in *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*, 2019, pp. 344–353.
- [48] W. Pourmajidi, L. Zhang *et al.*, "The challenging landscape of cloud monitoring," in *Knowledge Management in the Development of Data-Intensive Systems*. CRC Press, 2021, pp. 157–189.
- [49] K. Birman and T. Joseph, "Exploiting virtual synchrony in distributed systems," in *11th ACM Symposium on Operating Systems Principles*, ser. SOSP '87. Association for Computing Machinery, 1987, p. 123–138.
- [50] "Redis message broker — redis enterprise." [Online]. Available: <https://redis.io/solutions/messaging/>
- [51] S. Hoque and A. Miranskyy, "Architecture for analysis of streaming data," in *2018 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2018, pp. 263–269.
- [52] —, "Online and offline analysis of streaming data," in *2018 IEEE International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2018, pp. 68–71.
- [53] "Ibm cloud object storage." [Online]. Available: <https://www.ibm.com/products/cloud-object-storage>
- [54] "OpenTelemetry Documentation," 2024. [Online]. Available: <https://opentelemetry.io/docs/>
- [55] "Data Model · OpenZipkin." [Online]. Available: https://zipkin.io/page/s/data_model.html
- [56] "Devops solutions — ibm." [Online]. Available: <https://www.ibm.com/devops>
- [57] "Using toolchains — ibm cloud docs." [Online]. Available: <https://cloud.ibm.com/docs/ContinuousDelivery?topic=ContinuousDelivery-toolchains-using>
- [58] S. Ligu, *Effective monitoring and alerting*. O'Reilly Media, Inc., 2013.
- [59] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [60] M. Abadi, A. Agarwal *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [61] A. L. Maas, A. Y. Hannun *et al.*, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. icml*, vol. 30, no. 1. Atlanta, GA, 2013, p. 3.
- [62] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.
- [63] G. K. Karagiannidis and A. S. Lioumpas, "An improved approximation for the gaussian q-function," *IEEE Communications Letters*, vol. 11, no. 8, pp. 644–646, 2007.
- [64] A. Stolwijk, H. Straatman, and G. Zielhuis, "Studying seasonality by using sine and cosine functions in regression analysis," *Journal of Epidemiology & Community Health*, vol. 53, no. 4, pp. 235–238, 1999.
- [65] C. M. Bishop, "Pattern recognition and machine learning," New York, 2006.
- [66] C. O'Neil and R. Schutt, *Doing data science: Straight talk from the frontline*. O'Reilly Media, Inc., 2013.
- [67] A. Hrusto, E. Engström, and P. Runeson, "Optimization of anomaly detection in a microservice system through continuous feedback from development," in *Proceedings of the 10th IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems*, 2022, pp. 13–20.
- [68] —, "Towards optimization of anomaly detection in devops," *Information and Software Technology*, vol. 160, p. 107241, 2023.
- [69] A. Hrusto, P. Runeson, and M. C. Ohlsson, "Autonomous monitors for detecting failures early and reporting interpretable alerts in cloud operations," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, 2024, pp. 47–57.
- [70] C. Wohlin, P. Runeson *et al.*, *Experimentation in Software Engineering*, ser. Computer Science. Springer Berlin Heidelberg, 2012.
- [71] R. Yin, *Case Study Research: Design and Methods*, ser. Applied Social Research Methods. SAGE Publications, 2009.

- [72] R. J. Wieringa and M. Daneva, “Six strategies for generalizing software engineering theories,” *Science of computer programming*, vol. 101, pp. 136–152, 2015.
- [73] J.-l. Gailly and M. Adler, “Gnu gzip,” *GNU Operating System*, 1992.
- [74] M. Raasveldt and H. Mühleisen, “Duckdb: an embeddable analytical database,” in *2019 International Conference on Management of Data*, 2019, pp. 1981–1984.
- [75] “Apache parquet,” Apache Software Foundation, 2024. [Online]. Available: <https://parquet.apache.org>
- [76] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: http://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf
- [77] R. Fielding and J. Reschke, “Hypertext transfer protocol (http/1.1): Semantics and content,” Request for Comments: 7231, Internet Engineering Task Force, 2014, accessed: 2024-10-06. [Online]. Available: <https://tools.ietf.org/html/rfc7231>
- [78] *ISO 8601 — Date and time format*, International Organization for Standardization Std., 2004. [Online]. Available: <https://www.iso.org/iso-8601-date-and-time-format.html>

APPENDIX A DATA PREPROCESSING

The collected data undergoes the following “mutations”: (1) filtering, (2) aggregation, (3) transformation, and (4) masking. Each step is outlined below.

A. Data Filtering

The IBM Cloud CONSOLE logs and telemetry traffic are monitored, and the relevant telemetry records are filtered based on criteria defined by IBM DevOps teams. The filtered data are stored in compressed gzip [73] archives in COS. This reduces data points from thousands to approximately 200 per minute per data center, optimizing storage and processing needs.

B. Data Aggregation

The filtered data are aggregated into 5-minute intervals, aligned with astronomical time. Each interval (e.g., 0–5 minutes) groups incoming requests by rounding them to the start of the time range. The choice of a 5-minute interval is based on feedback from the IBM Ops team, as this duration smooths out noise and enhances the data’s usability. Eight aggregation functions — minimum, maximum, median, average, count, standard deviation, skewness, and kurtosis — are applied to prepare the data, specifically the response time (measured in milliseconds), for further analysis.

C. Data Transformation

The telemetry data, originally in JSON format, is ingested into DuckDB [74], a database management system optimized for efficient data analysis. Aggregated statistics, such as mean and median, are extracted from the JSON and then unpivoted into database columns. We share the resulting dataset in Apache Parquet format [75] (due to its efficiency in storing and accessing large datasets).

D. Dataset Masking

To ensure privacy and comply with IBM’s security policies, sensitive fields such as location, host, and endpoint (see Appendix B-A for details of the fields) are masked. These are replaced with obfuscated values like `datacenter1`, `component5`, and `endpoint8`.

TABLE IV
NUMBER OF COLUMNS FOR DIFFERENT GROUPS OF HTTP STATUS CODES IN THE PIVOTED VERSION OF THE DATASET.

2XX Successful	3XX Redirection	4XX Client Errors	5XX Server Errors	-1 Non-HTTP
52 288	4104	32 680	19 248	9128

APPENDIX B DATASET CHARACTERISTICS

A. Aggregated telemetry

The resulting dataset contains aggregated telemetry for 39 365 5-minute intervals. The data is provided in an unpivoted format in the file `unpivoted_data.parquet`, which contains 413 241 248 rows and 9 columns:

- **interval_start**: The start time of a 5-minute interval, represented in Epoch/Unix time format.
- **location**: 7 distinct values, representing data center ID.
- **kind**: 2 distinct values, namely `CLIENT` and `SERVER`, corresponding to the communication type.
- **host**: 54 distinct host IDs.
- **method**: 7 distinct REST API [76], [77] methods (e.g., `GET` or `POST`).
- **statusCode**: 30 distinct HTTP response status codes [77] (e.g., 200 or 500) and 1 non-HTTP response status code (namely, -1). The count of columns for different groups of HTTP status codes in the pivoted version of the dataset is given in Table IV.
- **endpoint**: 1001 distinct API endpoint IDs.
- **aggregated_stats_name**: One of eight aggregate statistical functions (detailed in Section III-C).
- **aggregated_stats_value**: Contains the values corresponding to the aggregate statistical functions.

We also provide a pivoted version of the dataset in the file `pivoted_data.parquet`, which contains 39 365 rows and 117 449 columns. Each row represents a specific time interval.

The first column is `interval_start`, and the remaining column names are following the template shown in Figure 6. The values in these columns represent the corresponding `aggregated_stats_value`.

Approximately 91% of the cells in the pivoted dataset contain null values, due to inactivity or low activity⁸ for a particular combination of location, kind, host, method, `statusCode`, endpoint, and aggregated statistical function.

B. Anomaly Windows

Details of annotating anomaly windows are given in Section III-D. The file `anomaly_windows.csv` contains ground truth data, listing the time intervals when the CONSOLE experienced anomalies. There are 25 anomalies in total. The file includes the following columns:

⁸For example, calculating the standard deviation requires at least two data points, so if there are fewer, the value is null.

Template: {location}_{kind}_{host}_{method}_{statusCode}_{endpoint}_{aggregated_stats_name}
Example: datacenter1_CLIENT_component10_GET_200_endpoint865_count

Fig. 6. Pivot dataset column template and column name example.

- **number:** A unique identifier for each anomaly.
- **anomaly_start:** The start time of the anomaly in ISO 8601 format [78] (e.g., 2024-02-02 10:22:00-0500).
- **anomaly_end:** The end time of the anomaly in ISO 8601 format.
- **anomaly_source:** The source of the anomaly. We assign numerical IDs to each source: 1 refers to “Issue Tracker”, 2 refers to “Instant Messenger”, and 3 refers to “Test Log”; see Section III-D for details.

C. The CONSOLE Instances Downtime

As mentioned in Section III-A, instances of the Console may be temporarily removed from or added to rotation in specific data centers. Note that removing an instance from rotation does not always indicate an anomaly; it could be a planned event, such as routine hardware or software maintenance. The file `location_downtime.csv` provides details on when these events occur. In total, 93 events have been recorded in this file. This information can be useful for enhancing anomaly detection features or refining existing ones. The file includes the following columns:

- **location:** The ID of the data center.
- **downtime_start:** The start time of the downtime, in ISO 8601 format.
- **downtime_end:** The end time of the downtime, in ISO 8601 format.